

Week 4: Loop Statements

General Instructions:

1. Remove existing directories and create a fresh directory of your own. Write all the programs inside the newly created directory.
2. Remove the directory that you have created at the end of your Lab session.
3. Inform one of the TAs before you leave the Lab.

In the previous week, we have learnt the followings

1. What is a statement? We have seen two types of statement – simple statement/expression, and comments
2. Variable declaration
3. Different types of datatypes
4. The printf and scanf functions
5. The scope of a variable
6. The conditional Statements

Loop statements special types of **composite statement where a block of statements is executed multiple times till a defined condition is satisfied**. Some types of loop statements supported in C language are

- a. The for loop
- b. The while loop
- c. The do-while loop
- d. The label and goto statement (label statement can also be used for loop)

1. What is a loop statement?

A Loop statement is used to execute a block of code several times according to the condition given in the loop. It means it executes the same code multiple times so it reduces the size of a program from repeated appearing a code segment.

Let us say, we wish to print a message “hello world” 5 times”. One simple way of doing it is, writing printf() statement 5 times.

```
#include<stdio.h>
main(){
    printf("Hello World");
    printf("Hello World");
    printf("Hello World");
    printf("Hello World");
    printf("Hello World");
}
```

5 times is small and it is feasible. What if I need to print it for 100 times, or 1000 times. Is it encouraging for a programmer to write the same code of program segment 100 times or 1000 times. This is where a loop comes handy. It not only reduces the size of the program, but also help in minimizing potential programmer fatigue related errors.

2. The for loop Statement

The syntax of a **for loop** has the following format.

```
//      Part A      Part B      Part C
for (initialization; condition; increment/decrement) {
    // Part D
    // statements to be repeated
}
```

Note that there are four parts in the for loop statement, as defined below.

Part A: Initialization statement

Part B: Condition statement

Part C: Increment/Decrement statement

Part D: Body of the **for loop** that you want to repeat, if the part B is satisfied.

The parts A, B and C are separated by semicolon (;). The execution of a for loop will be performed in the following sequence.

First, the statement(s) in the initialization part will be executed i.e., Part A will be executed.

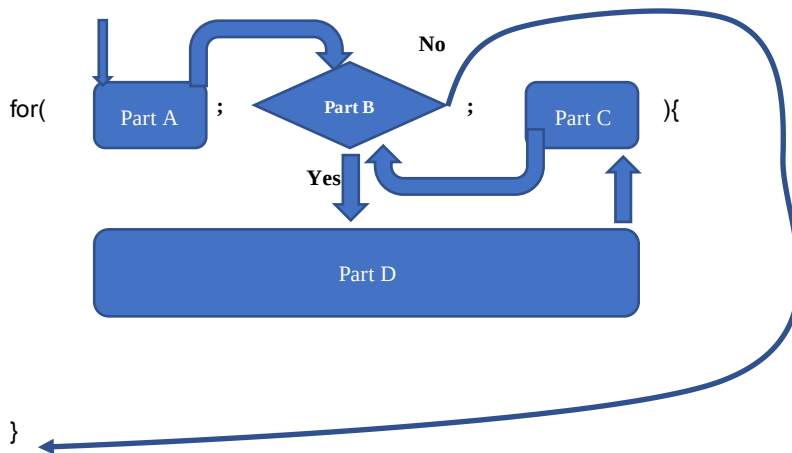
Second, the statements in the condition part (part B) will be executed. The statement in the Part B should be a conditional statement i.e., returns a Boolean value (true/false or 1/0).

Third, if the statement in Part B returns **true**, the statements in the Part D will be executed. Otherwise, the execution exits from the loop. That is, Part D will be executed if Part B returns true.

Forth, if Part D is executed, it then execute Part C, Then it goes to step second to execute Part B and repeat steps thirds and forth.

Note that Part A executes only one time.

Following figure illustrates the above sequence.



Examples:

a. Print Hello World for n times

```
#include<stdio.h>
main(){
    int i; n;

    printf("Enter the number, that many times you want to repeat:");
    scanf("%d", &n);

    for(i = 0; i < n; i = i + 1){

        printf("Hello World \n");

    }
}
```

If you enter 5 as the value of n, the above for loop will print the message 5 times. Let us decode and understand the sequence of execution.

Sequence	Executed Statement	Part	Remarks (assume that n is 5)
1	i = 0	Part A	The initialization statement is executed. That is, the variable i is initialized to 0
2	i < n	Part B	if (i < 5) is executed. As i=0, it returns true .
3	printf("Hello World \n");	Part D	Print the message
4	i = i + 1	Part C	Increment the value of i to 1
5	i < n	Part B	if (i < 5) is executed. As i=1, it returns true .
6	printf("Hello World \n");	Part D	Print the message
7	i = i + 1	Part C	Increment the value of i to 2
8	i < n	Part B	if (i < 5) is executed. As i=2, it returns true .
9	printf("Hello World \n");	Part D	Print the message
10	i = i + 1	Part C	Increment the value of i to 3
11	i < n	Part B	if (i < 5) is executed. As i=3, it returns true .
12	printf("Hello World \n");	Part D	Print the message
13	i = i + 1	Part C	Increment the value of i to 4
14	i < n	Part B	if (i < 5) is executed. As i=4, it returns true .
15	printf("Hello World \n");	Part D	Print the message
16	i = i + 1	Part C	Increment the value of i to 5
17	i < n	Part B	if (i < 5) is executed. As i=5, it returns False . And, the loop exits.

Therefore, the total number of statements executed in the above loop is 17.

b. Print first n natural numbers

```
#include<stdio.h>
main(){
    int i; n;

    printf("Enter n:");
    scanf("%d", &n);

    for(i = 0; i < n; i = i + 1){

        printf("%d \n", i+1);

    }
}
```

c. Print odd natural numbers smaller than n

```
#include<stdio.h>
main(){
    int i; n;

    printf("Enter n:");
    scanf("%d", &n);

    for(i = 0; i < n; i = i + 1){

        if(i % 2) // for odd i%2 returns 1, which is equivalent to True

            printf("%d \n", i);

    }
}
```

d. Print prime numbers (1 to 100)

```
#include <stdio.h>
main(){

    int i, N, count;
    for(N = 1; N <= 100; N++){
        count = 0;
        for (i = 2; i <= N/2; i++){
            if(N%i == 0){
                count++;
                break;
            }
        }
        if(count == 0 && N != 1){
            printf(" %d ", N);
        }
    }
}
```

In the above program, an operator (++) has been introduced. It is known as unary increment by 1. Two of the commonly used unary operators are ++ and --. The ++ is unary increment by 1, and -- is unary decrement by 1. These two operators are illustrated more in details below.

- a. **The k++ or ++k:** The k here represents a variable. The k++ or ++k mean, the value of the variable k is increment by 1. However, k++ is known as post increment, whereas ++k means pre-increment. Let us consider the following two examples.

```
#include<stdio.h>
main(){
    int a=1, b;
    b=++a;
    printf("%d %d", b, a);
}
```

In the above example, the value of b will be 2 and the value of a will be 2. It means, the value of the variable a is incremented before executing the assignment operator (=) i.e., pre-increment with regards **to its associated operator**.

```
#include<stdio.h>
main(){
    int a=1, b;
    b=a++;
    printf("%d %d", b, a);
}
```

In the above example, the value of b will be 1, and the value of a will be 2. It means, the value of the variable a is incremented after executing the assignment operator (=) i.e., post-increment to its associated operator.

Also note that unary operators have right to left associativity i.e., execution starts from right to left.

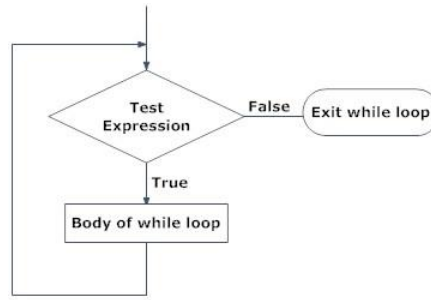
Given the initial values of a=1, check the value of b for the expression b=(a++)+(a++); and b=(++a)+(++a); Try to understand the output. It is left as an exercise for you to ponder upon.

- b. Similarly, k- - and - -k represent post decrement and pre decrement respectively.

3. The while loop:

While loop executes the code until the condition is false. Its syntax is defined below.

```
while(condition){
// statements to be repeated
}
```



Example:

- a. Print Hello World for n times

```
#include<stdio.h>
main(){
    int i; n;

    printf("Enter the number, that many times you want to repeat: ");
    scanf("%d", &n);

    i=0;
    while(i<n){

        printf("Hello World \n");
        i++; // Indrement or decrement needs to be included to ensure
            //termination. Otherwise, the loop will go in infinite iteration
    }
}
```

Try converting the above for loops to its corresponding while equivalent

4. The do .. while loop:

It is similar to while loop, but the comparison is at the end. It also executes the code until condition is false. In this at least once, code is executed whether condition is true or false but this is not the case with while. While loop is executed only when the condition is true.

```
do{

// Statements to be repeated until the condition below is false

}while(condition);
```

Example:

- a. Print Hello World for n times

```
#include<stdio.h>
main(){
    int i; n;

    printf("Enter the number, that many times you want to repeat: ");
    scanf("%d", &n);

    i=0;
```

```

do {

    printf("Hello World \n");
    i++;

}while(i<=n) //see the difference in the conditional statement
}

```

Try converting the above for loops to its corresponding do-while equivalent

5. **Label and go to Statement:**

A statement in C can be assigned a label, and execution control can also be jumped from one place to another using label and goto respectively, without a condition.

Syntax.

Syntax1		Syntax2

goto label;		label:
.		.
.		.
.		.
label:		goto label;

Example: Check if num is odd or even

```

#include <stdio.h>

main()
{
    int num = 26;
    if (num % 2 == 0)
        // jump to even
        goto even;
    else
        // jump to odd
        goto odd;

    even:
        printf("%d is even", num);
        return 1;
    odd:
        printf("%d is odd", num);
        return 1;
}

```

Using goto and label statement, you can create a loop. It has been left as an exercise. Try printing hello world using goto and label statements. For such statement, you should mind the terminating statements.